

第11章 反射与动态特性

内省 · 属性拦截 · 动态造类 · 猴子补丁

本章目标:讲透 Python 的反射能力:dir/type/inspect 程序自省、__getattr__/__setattr__ 属性拦截、type 动态造类、importlib 动态加载,以及猴子补丁的双刃剑。学完你能写出调试器、序列化器、插件加载器这类"元工具",并清楚反射的边界与风险。

直觉起点:C 程序在编译后只是一堆指令与符号表,程序无法问自己"我有哪些函数";Python 的对象带着完整元信息运行,任何对象都能被追问"你是什么、你有什么、你能做什么"。反射就是这种自我审视的能力——它在 C 里几乎不存在,是 Python 动态性的核心。

前置要求:第6章面向对象(属性与类)、第10章类型与元编程(类创建机制)。

本章路线:第一阶段(1 小时)读 §11.1–§11.2,掌握内省工具;第二阶段(2 小时)读 §11.3–§11.5,理解属性拦截与动态构造;第三阶段(2 小时)读 §11.6–§11.8,认识猴子补丁并完成章末实验。

本章知识导图

- └─ 11.1 内省工具 — dir / type / vars / getattr
- └─ 11.2 inspect — 签名、源码、类层次
- └─ 11.3 属性拦截 — __getattr__ / __setattr__
- └─ 11.4 动态造类 — type() 与运行时方法
- └─ 11.5 动态导入 — importlib 与插件加载
- └─ 11.6 猴子补丁 — 威力与代价
- └─ 11.7 反射序列化 — 通用对象转字典
- └─ 11.8 生产实践 — 反射的边界

§11.1 内省工具:dir / type / vars / getattr

内省四件套覆盖"看有什么、是什么、取什么"的全部需求:dir(obj) 列出所有属性名;type(obj) 得类型;vars(obj) 得实例的 __dict__;动态取属性用 getattr(obj, name),动态设属性用 setattr(obj, name, value):

```

class Config:
    def __init__(self) -> None:
        self.env = "prod"
        self.debug = False

    def reload(self) -> None:
        print("重新加载配置")

cfg = Config()
print(dir(cfg))          # ['__class__', ..., 'debug', 'env', 'reload']
print(vars(cfg))         # {'env': 'prod', 'debug': False}

# 按字符串动态取属性:配置驱动
key = "debug"
print(getattr(cfg, key)) # False
setattr(cfg, key, True)  # 等价于 cfg.debug = True
print(cfg.debug)         # True

# 不存在时给默认值,避免 AttributeError
print(getattr(cfg, "missing", "无此属性")) # 无此属性

```

C(静态内省)	Python(运行期内省)
sizeof/offsetof 只给内存布局	dir/vars 给出完整成员清单与当前值
符号表只在调试器里可见	程序自己就能枚举方法并调用
类型信息编译期固定	type() 运行期可得,且能判断 isinstance
字段增减要改结构体并重编译	setattr 随时加属性,无需重编译

getattr 的第三个参数是反射的标配动作——“可能不存在”的访问永远带默认值或 try/except,这是反射代码的第一纪律。dir 的输出注意区分:实例属性、类属性、双下划线魔法属性混在一起,过滤 `startswith("__")` 是常规操作。

内省的第一步永远是“看”,第二步才是“取”:先 dir 确认存在,再 getattr 读取——但生产上不要先查再取(两步之间有竞态),直接用带默认值的 getattr 一步到位。遍历对象的通用工具(调试器、序列化器)要把“实例属性、类属性、继承属性”三层分开处理,vars 只看实例层,dir 看全部。

§11.2 inspect:签名、源码与类层次

inspect 是标准库的深度内省模块:拿函数签名、看实现源码、遍历类层次,还能判断“这个对象是不是类/方法/协程”。调试器、文档生成器、测试框架都建立在它之上:

```
import inspect

def connect(host: str, port: int = 8080, *, timeout: float = 5) -> None:
    """建立连接"""
    ...

sig = inspect.signature(connect)
print(sig)                    # (host: 'str', port: 'int' = 8080, *, timeout: 'float' = 5)
for name, param in sig.parameters.items():
    print(name, param.kind, param.default)
# host POSITIONAL_OR_KEYWORD
# port POSITIONAL_OR_KEYWORD 8080

print(inspect.getsource(connect))    # 打印函数源码
print(inspect.isclass(Config), inspect.isfunction(connect))
```

生产中最常用的三个能力:signature 做参数校验与自动文档;getsource 排查"这个函数到底是谁的"(尤其装饰器包裹后);ismethod/isfunction 区分绑定方法、静态方法与普通函数——框架层统一处理"可调用对象"时必须用它们兜底。

inspect 还有一个高频用途:协程与生成器的判断(iscoroutine/isgenerator)。异步框架调度任务前必须分清"普通函数、生成器、协程",传错类型会在 await 处爆出难解的报错;用 inspect 预检,错误提前到调度点,信息也清晰得多。

§11.3 属性拦截: __getattr__ 与 __setattr__

Python 允许类拦截属性访问: __getattr__ 只在"正常查找失败"时调用(懒加载的天然实现); __setattr__ 在每次属性赋值时调用(校验与只读的强制实现); __getattribute__ 拦截一切读取(危险,慎用):

```
class LazyConfig:
    """懒加载配置:首次访问才真正读取文件"""

    def __init__(self, path: str) -> None:
        self._path = path
        self._loaded = False
        self._data = {}

    def __getattr__(self, name: str):
        """只在正常查找失败时触发:惰性加载"""
        if name == "data" and not self._loaded:
            self._data = {"env": "prod", "workers": 4} # 模拟读文件
            self._loaded = True
        return self._data[name] # 返回配置值

c = LazyConfig("config.json")
print(c.workers)                # 4 — 首次访问触发加载
print(c.env)                    # prod
print(c.workers)                # 4 — 已缓存,不再加载
```

误区: __setattr__ 里写 self.x = value 无限递归。__setattr__ 拦截一切赋值,在它内部再给 self.x 赋值会再次进入 __setattr__ 直到 RecursionError。正确姿势:用 object.__setattr__(self, name, value) 或操作

`self.__dict__` (与第10章描述符的存储模式同源)。同理, `__getattr__` 里访问 `self.xxx` 也可能绕回自身,先检查 `__dict__` 或使用前缀保护。

```
class Immutable:
    """ __setattr__ 强制只读:赋值一律拒绝"""

    def __init__(self, value: int) -> None:
        object.__setattr__(self, "value", value)

    def __setattr__(self, name: str, value) -> None:
        raise AttributeError(f"{name} 是只读属性")

obj = Immutable(42)
print(obj.value)           # 42
obj.value = 1              # AttributeError: value 是只读属性
```

三种拦截的选型:只做"缺属性时的兜底"用 `__getattr__` (幂等、安全);要强制校验或只读用 `__setattr__`; `__getattribute__` 拦截所有读取,性能与复杂度代价都高,99% 的场景不需要它。代理对象、mock、ORM 懒加载字段是这三个钩子的经典生产场景。

属性拦截的调试技巧:在 `__getattr__`/`__setattr__` 里加一行 `print(name, value)` 或日志,立刻看到"谁在读写什么"——排查魔法属性意外触发、框架注入字段这类玄学问题时,这个手段比断点还快。生产上把日志级别设为 `debug`,平时零成本,出问题有抓手。

§11.4 动态造类: `type()` 与运行时方法

`class` 语句在运行期等价于 `type(name, bases, namespace)` 调用(第10章元类已见);反过来,我们可以在运行期凭空构造一个类,再给实例动态绑定方法:

```

# 方式一:type 三参数动态造类
Point = type("Point", (object,), {
    "__init__": lambda self, x, y: (setattr(self, "x", x),
                                     setattr(self, "y", y)),
    "norm": lambda self: (self.x ** 2 + self.y ** 2) ** 0.5,
})
p = Point(3, 4)
print(p.norm())           # 5.0

# 方式二:给实例动态绑方法(需要 MethodType 绑定 self)
import types

class Calc:
    pass

def square(self, n: int) -> int:
    return n * n

c = Calc()
c.square = types.MethodType(square, c)  # 绑定后 self 自动传入
print(c.square(6))           # 36

```

动态造类的生产场景:数据驱动生成 DTO、按配置文件构造策略对象、测试里快速造替身。注意纪律:动态生成的类没有静态检查护航,mypy 对它们基本失明;能写静态类就写静态类,动态造类只用于"数量不可预知"的场景。

动态造类与静态类的关系,类似 C 里宏与函数的关系:宏(动态)灵活但不可调试,函数(静态)可读可查。一个实用的中间态是工厂函数:逻辑写在静态类里,工厂按配置选择并实例化——既有静态类的可读性,又有动态分发的灵活性,多数"动态需求"用工厂就够。

§11.5 动态导入:importlib 与插件加载

插件系统的另一条腿是"按名字加载模块"。importlib.import_module 把字符串变成可用的模块对象,配合 getattr 取其中的类——第10章的注册表与这里连成完整闭环:

```

import importlib
from pathlib import Path

def load_plugin(module_name: str, class_name: str):
    """从模块路径动态加载插件类"""
    module = importlib.import_module(module_name)
    cls = getattr(module, class_name)
    return cls()

def load_all_from(dir_path: Path) -> list:
    """扫描目录下所有 check_*.py,加载其中 main()"""
    plugins = []
    for f in dir_path.glob("check_*.py"):
        mod = importlib.import_module(f"plugins.{f.stem}")
        plugins.append(mod.main)
    return plugins

```

误区:动态加载失控。 import_module 的字符串来自配置文件或用户输入时,等于给了任意代码执行入口——恶意插件路径能加载任意模块。生产纪律:① 插件目录固定且受控,白名单校验模块名(正则 `^[a-z_]+$`);② 加载失败 (ImportError/AttributeError)要捕获并给出"哪个插件、缺什么"的清晰报错;③ 反射加载的代码视同第三方代码,一样要过评审与测试。

动态导入的另一面是"导入期副作用":模块顶层代码(注册、连库、打印)在 import 时就执行。插件系统要约定"插件模块只定义,不执行",main/run 由框架显式调用——把"加载"与"执行"分离,是插件架构的稳定性基石。

§11.6 猴子补丁:威力与代价

猴子补丁是运行期替换对象的属性/方法——给第三方库打补丁、测试替身、A/B 灰度都靠它。经典用例:修复第三方库 bug、给库方法加缓存、测试中替换网络层:

```

import time
import requests

# 场景:给 requests.get 统一加日志与超时兜底
_original_get = requests.get          # 先存原函数!

def patched_get(url, **kwargs):
    kwargs.setdefault("timeout", 10) # 没传超时的请求统一兜底
    print(f"[patched] GET {url}")
    return _original_get(url, **kwargs)

requests.get = patched_get            # 打补丁:全局生效

resp = requests.get("https://example.com")

```

C(函数覆盖)	Python(猴子补丁)
弱符号覆盖在链接期,静态、全局	运行期任意对象任意属性,可恢复可撤销
dlsym 拿符号表地址,晦涩	直接赋值 <code>mod.func = new</code> ,一行完成
覆盖后原函数丢了(需手动保存地址)	先存原引用即可随时还原(如上面的 <code>_original_get</code>)
生效范围:整个进程	生效范围:整个进程——同样全局,且难以追溯

误区:猴子补丁当常规手段。它全局生效、隐式修改、难以调试——被补丁的模块在别处可能依赖原行为,测试间的补丁还会互相污染。纪律:① 只补第三方库,不补自己的代码(自己的代码直接改);② 补丁代码集中在一个模块,写明"补什么、为什么";③ 测试替身优先用 `unittest.mock.patch`(自动还原),不用手写赋值;④ 生产补丁必须有日志与监控,升级库后第一时间复查补丁是否还必要。

补丁的"保质期"问题常被忽略:库升级后补丁可能已被官方修复,旧补丁反而引入双重重试、日志噪音等新 bug。生产规范:每个补丁文件头写明"创建日期、上游 issue 链接、移除条件",升级依赖时把补丁清单过一遍,该删就删。

§11.7 反射序列化:通用对象转字典

把任意对象转成 dict/JSON(序列化器、日志、调试工具的核心)是反射最实用的场景之一:遍历实例的 `__dict__`,递归转换基本类型,处理循环引用:

```
def to_dict(obj, seen: set | None = None) -> object:
    """把任意对象递归转成可 JSON 序列化的结构"""
    if seen is None:
        seen = set()
    if isinstance(obj, (str, int, float, bool)) or obj is None:
        return obj
    if isinstance(obj, (list, tuple, set)):
        return [to_dict(x, seen) for x in obj]
    if isinstance(obj, dict):
        return {str(k): to_dict(v, seen) for k, v in obj.items()}
    if id(obj) in seen:
        # 循环引用兜底:返回引用标记
        return {"__ref__": id(obj)}
    seen.add(id(obj))
    result = {}
    for key, value in vars(obj).items():
        # 反射:取所有实例字段
        result[key] = to_dict(value, seen)
    return result

class User:
    def __init__(self, name: str, tags: list) -> None:
        self.name = name
        self.tags = tags
        self.friend = None

u = User("李雷", ["vip", "cn"])
u.friend = User("韩梅梅", ["vip"])
import json
print(json.dumps(to_dict(u), ensure_ascii=False, indent=1))
```

这段代码一次覆盖了本章多数主题:isinstance 分派、vars 内省、seen 集合防循环引用。生产级序列化器(jsonable 库、pydantic)在其上增加了类型白名单与 __reduce__ 协议,但骨架就是反射——理解它,你就理解了 ORM 与 RPC 框架的一半。

§11.8 生产实践:反射的边界

反射是"元工具"的燃料,但有三条不可逾越的边界:① **可读性边界**——反射代码无法静态搜索"谁调用了我",IDE 跳转失效,项目里反射使用率超过 5% 就要警惕;② **性能边界**——getattr 动态查找比直接属性访问慢一个量级,热路径禁止反射;③ **安全边界**——字符串驱动的反射(getattr(obj, user_input))与 eval/exec 一样是注入面,永远白名单。

工程建议 1:反射的调试神器是 __repr__ 与 inspect:给所有业务类写 __repr__,排查"这个对象到底是什么"时一个 print 全清楚;怀疑方法来源时 inspect.getsource 直接看源码,比猜快得多。

工程建议 2:优先用"标准库提供的反射"而不是自己写:dataclasses.fields、inspect.signature、typing.get_type_hints 都是官方打磨过的实现;自己遍历 __dict__ 做参数匹配,边界情况(继承、描述符、插槽类)会让你踩不完。

本章速查表

场景	写法
列出成员	dir(obj) 过滤 startswith("__")
取实例字段	vars(obj) / obj.__dict__
动态取属性	getattr(obj, name, default)
动态设属性	setattr(obj, name, value)
函数签名	inspect.signature(func)
函数源码	inspect.getsource(func)
缺属性兜底	def __getattr__(self, name):
赋值拦截	def __setattr__(self, name, value): ,内部用 object.__setattr__
动态造类	type(name, bases, namespace)
实例绑方法	types.MethodType(func, instance)
按名加载模块	importlib.import_module("pkg.mod")
打补丁	先存原引用,再 mod.func = new ,用后还原
测试替身	unittest.mock.patch("pkg.mod.func") ,自动还原
防循环引用	seen 集合记录 id,命中返回标记
反射纪律	热路径禁用;字符串驱动必须白名单
优先现成	dataclasses.fields / typing.get_type_hints 优先于手写

最小必会清单

1. 能用 dir/vars/getattr/setattr 完成"配置驱动"的动态属性读写

2. 能用 `inspect` 拿到函数签名与源码,并判断对象的类别
3. 能写出不递归的 `__getattr__` (懒加载)与 `__setattr__` (只读/校验)
4. 能用 `type()` 动态造类、`MethodType` 给实例绑方法,并说出适用边界
5. 能用 `importlib` 实现插件加载,并说出两个安全纪律
6. 能解释猴子补丁的原理、典型场景与三条使用纪律

自测题

Q

Q1. `getattr(obj, "x")` 与 `obj.x` 有什么区别?什么场景必须用 `getattr`?

答案:功能相同,区别在属性名是否已知:`obj.x` 的属性名写死在代码里,`getattr` 接受字符串变量。必须用 `getattr` 的场景:属性名来自配置/数据(反射)、需要默认值(`getattr(obj, "x", 0)`)、遍历未知结构。已知属性名直接用点号——更可读、更快、可被静态检查。

Q

Q2. `__getattr__` 与 `__getattribute__` 的区别?为什么说后者危险?

答案:`__getattr__` 只在正常查找失败后调用(兜底、幂等);`__getattribute__` 对每次属性读取无条件调用,包括类属性、方法、魔法属性——内部任何 `self.xxx` 访问都再次触发它,极易无限递归,且性能开销大。生产 99% 场景用 `__getattr__`,只有实现代理、安全拦截等少数场景才碰 `__getattribute__`。

Q

Q3. `__setattr__` 里 `self.name = value` 为什么 `RecursionError`?怎么写才对?

答案:`__setattr__` 拦截一切赋值,`self.name = value` 又一次触发自身,无限递归。正确写法:`object.__setattr__(self, name, value)` 绕过拦截直接设置,或写 `self.__dict__[name] = value`。`__init__` 里的首次赋值同样要走这两条路,否则初始化即爆栈。

Q

Q4. 猴子补丁的典型场景有哪些?为什么"先保存原函数引用"是好习惯?

答案:典型场景:给第三方库打 bug 修复、统一包装(加超时/日志/缓存)、测试替身。保存原引用可随时还原补丁(尤其测试结束)、可在包装函数里调用原实现(装饰器模式)、可对比新旧行为。不保存则补丁不可逆,排障时无法对照,是生产事故隐患。

Q

Q5. 为什么说"字符串驱动的反射"是安全风险?如何防护?

答案:`getattr(obj, user_input)` 或 `import module(user_input)` 把用户输入变成代码路径/模块加载——恶意输入可访问任意属性、加载任意模块,等价于代码注入。防护:① 输入白名单校验(正则/集合匹配);② 用映射表翻译"允许的键"而非直接透传字符串;③ 反射加载失败时只暴露受控的错误信息,不泄露内部路径。

Q

Q6. 写通用序列化器时,为什么需要 `seen` 集合?不加会怎样?

答案:对象图可能有环(双向引用、自引用):`A.friend = B` 且 `B.friend = A`,递归遍历将无限循环直到 `RecursionError`。`seen` 集合记录已访问对象的 `id`,再次遇到时返回引用标记(如 `{"_ref_": id}`)或直接跳过,把环变成有限结构。真实序列化器(JSON 默认)遇到环会抛错,定制时也要显式决定环的表示。

章末实验题:通用遍历器与反射序列化

实验 11•1 对象遍历打印器与反射序列化(覆盖:\$11.1–\$11.7)

需求:① 写一个 `dump(obj, indent=0)` 函数:递归遍历任意对象的属性,以缩进树状打印"属性名:值",基本类型直接打印,容器逐元素,自定义对象进一层缩进;② 处理循环引用(`seen` 集合,打印 [循环引用] 标记);③ 只打印公开成员(跳过 `__` 开头,跳过方法/类);④ 写一个 `to_jsonable(obj)`:把任意对象转成可 JSON 序列化的结构(复用 \$11.7 思路,支持 list/dict/自定义类/基本类型);⑤ 用 `User/Order` 两层对象验证输出,并对比 `json.dumps` 结果;⑥ 用 `__repr__` 配合 `inspect` 输出对象摘要。

覆盖知识点:内省工具 `vars/dir/getattr`(\$11.1)/`inspect`(\$11.2)/属性拦截(\$11.3)/反射序列化(\$11.7)。

实验环境:Python 3.10+,单文件 `objtool.py`。

```

# objtool.py — 对象遍历打印机与反射序列化(覆盖第11章全部知识点)
import inspect
import json

def _is_public_member(value) -> bool:
    """跳过方法、魔法属性与私有成员"""
    if inspect.ismethod(value) or inspect.isfunction(value):
        return False
    return True

def dump(obj, indent: int = 0) -> None:
    """树状打印对象结构:属性名: 值"""
    prefix = " " * indent
    if isinstance(obj, (str, int, float, bool)) or obj is None:
        print(f"{prefix}{obj!r}")
        return
    if isinstance(obj, (list, tuple)):
        print(f"{prefix}[{type(obj).__name__}]")
        for item in obj:
            dump(item, indent + 1)
        return
    if isinstance(obj, dict):
        print(f"{prefix}{{dict}}")
        for k, v in obj.items():
            print(f"{prefix} {k}: ", end="")
            dump(v, indent + 2)
        return
    print(f"{prefix}<{type(obj).__name__}>")
    for name, value in vars(obj).items():
        if name.startswith("__"):
            continue
        print(f"{prefix} {name}: ", end="")
        dump(value, indent + 2)

def to_jsonable(obj, seen: set | None = None) -> object:
    """任意对象转可 JSON 序列化结构,循环引用安全"""
    if seen is None:
        seen = set()
    if isinstance(obj, (str, int, float, bool)) or obj is None:
        return obj
    if isinstance(obj, (list, tuple)):
        return [to_jsonable(x, seen) for x in obj]
    if isinstance(obj, dict):
        return {str(k): to_jsonable(v, seen) for k, v in obj.items()}
    if id(obj) in seen:
        return {"__ref__": id(obj)}
    seen.add(id(obj))
    result = {}
    for name, value in vars(obj).items():
        if name.startswith("__"):
            continue

```

```
    result[name] = to_jsonable(value, seen)
return result
```

```
class User:
    def __init__(self, name: str, age: int) -> None:
        self.name = name
        self.age = age
        self.orders = []

    def __repr__(self) -> str:
        return f"User({self.name}, {self.age})"

class Order:
    def __init__(self, oid: str, amount: float) -> None:
        self.oid = oid
        self.amount = amount

def main() -> None:
    alice = User("alice", 30)
    alice.orders = [Order("A001", 199.0), Order("A002", 58.5)]
    alice.friend = alice          # 自引用:验证循环引用处理

    print("== dump 输出 ==")
    dump(alice)

    print("== to_jsonable 输出 ==")
    print(json.dumps(to_jsonable(alice),
                     ensure_ascii=False, indent=1))

    print("== inspect 摘要 ==")
    print(f"alice 类型: {type(alice).__name__}")
    print(f"User 方法: {[m for m in dir(User) if not m.startswith('_')]}")

if __name__ == "__main__":
    main()
```

设计说明:① dump 与 to_jsonable 共用"vars + 过滤魔法成员"的遍历骨架,一个面向人读、一个面向机器读,体现反射工具的分工;② 循环引用统一用 seen 集合拦截,打印与序列化都给出可见标记;③ 过滤规则用 inspect 判断可调用对象,避免把方法当字段打印;④ 自引用示例(alice.friend = alice)直接验证防环逻辑;⑤ 输出均可通过测试断言校验(如 to_jsonable 结果包含 friend 的 __ref__ 标记)。

下一章预告:第12章 装饰器与函数式编程——装饰器本质、functools.wraps、带参数与类装饰器、contextmanager 与 lru_cache、自写计时/重试/缓存。让横切逻辑从代码里蒸发。